

Project Title

AI-Powered Network Monitoring, Automation & Reliability Platform

A production-style Python software platform that helps network teams **monitor devices, automate configurations, detect failures, generate reports, and improve system reliability.**

This project is strongly aligned with the recruiter's description: **network monitoring, automation, device configuration, reporting, debugging, and reliability engineering.**

Project Description

This application acts like a mini version of an enterprise network control center, inspired by platforms such as **Cisco Catalyst Center**, which provides centralized network management, automation, assurance, troubleshooting, APIs, and reporting for large-scale networks.

The system will simulate routers, switches, access points, servers, and edge devices. These simulated devices continuously send telemetry such as:

```
{  
  "device_id": "router-12",  
  "cpu": 86,  
  "memory": 74,  
  "latency": 210,  
  "packet_loss": 4.8,  
  "status": "warning"  
}
```

Your platform will collect this data, detect problems, automate recovery actions, manage configurations, and show everything in a real-time dashboard.

Target Audience

1. Network Operations Teams

They need to monitor hundreds of devices and quickly identify failures.

2. Telecom / Private 5G Companies

They manage routers, edge nodes, gateways, and distributed network infrastructure.

3. Enterprises / Universities / Hospitals

They need centralized visibility into switches, Wi-Fi access points, users, and branch networks.

4. SRE / Infrastructure Teams

They need reliability metrics, incident tracking, alerting, and automation.

Main Use Case

Imagine a company has 100 routers and switches.

Suddenly:

- latency increases,
- packet loss rises,
- one router CPU goes above 90%,
- a configuration change causes instability.

Your system will:

1. Detect the abnormal behavior.
2. Mark the device as unhealthy.
3. Create an incident.
4. Send a real-time dashboard alert.
5. Run an automated recovery workflow.
6. Compare device config with approved config.
7. Generate an incident report.

8. Store the event for future analytics.
-

Complete Feature List

1. Device Simulator

Software-only simulated devices.

Features:

- simulate routers, switches, APs, servers, edge nodes
 - generate CPU, memory, latency, packet loss, bandwidth
 - simulate device failure
 - simulate config drift
 - simulate network outages
-

2. Real-Time Network Monitoring

Features:

- device heartbeat tracking
 - uptime monitoring
 - latency monitoring
 - packet loss monitoring
 - bandwidth usage
 - CPU/memory usage
 - online/offline status
 - health score per device
-

3. AI Anomaly Detection

Detect:

- abnormal latency spikes
- unusual traffic patterns
- high packet loss

- CPU/memory anomalies
- possible outage risk
- unstable device behavior

Models:

- Isolation Forest
 - Random Forest
 - LSTM later as advanced version
-

4. Automation Engine

This is one of the most important features.

Automation workflows:

- auto-restart failed device agent
 - auto-mark device as degraded
 - auto-create incident
 - auto-send alert
 - auto-run config validation
 - auto-trigger rollback if config drift is detected
-

5. Device Configuration Management

Features:

- store device configs
- version configs
- validate configs
- detect config drift
- rollback to previous config
- push template-based config to simulated devices

Example config:

```
device_id: router-12
hostname: edge-router-12
interface: eth0
vlan: 20
routing_protocol: ospf
```

firewall_enabled: true

6. Config Drift Detection

Detect when current config differs from approved baseline.

Example:

Expected VLAN: 20

Actual VLAN: 30

Status: Drift Detected

Action: Rollback Recommended

7. Incident Management

Features:

- create incidents automatically
 - assign severity
 - track incident timeline
 - store root cause
 - mark resolved/unresolved
 - attach affected devices
 - generate post-incident summary
-

8. Real-Time Dashboard

Dashboard pages:

- network overview
- device health table
- topology map
- live alerts
- incident feed
- device details page
- config management page
- reports page

9. Reporting System

Reports:

- uptime report
- device health report
- incident report
- SLA report
- configuration compliance report
- anomaly summary report

Export formats:

- PDF
 - CSV
 - JSON
-

10. Reliability Score Engine

Each device gets a reliability score based on:

- uptime
- incident count
- packet loss
- latency
- anomaly frequency
- config stability

Example:

router-12 reliability score: 72/100

Status: Needs attention

11. Alerting System

Alerts through:

- dashboard notification

- email
- Slack-style webhook
- SMS later as optional

Alert types:

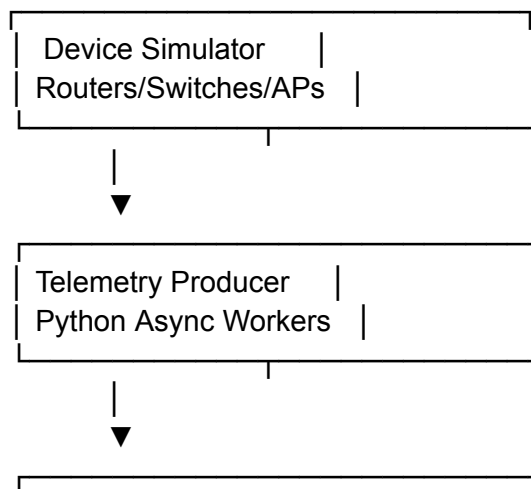
- critical outage
 - warning latency
 - packet loss
 - config drift
 - service recovery
 - automation completed
-

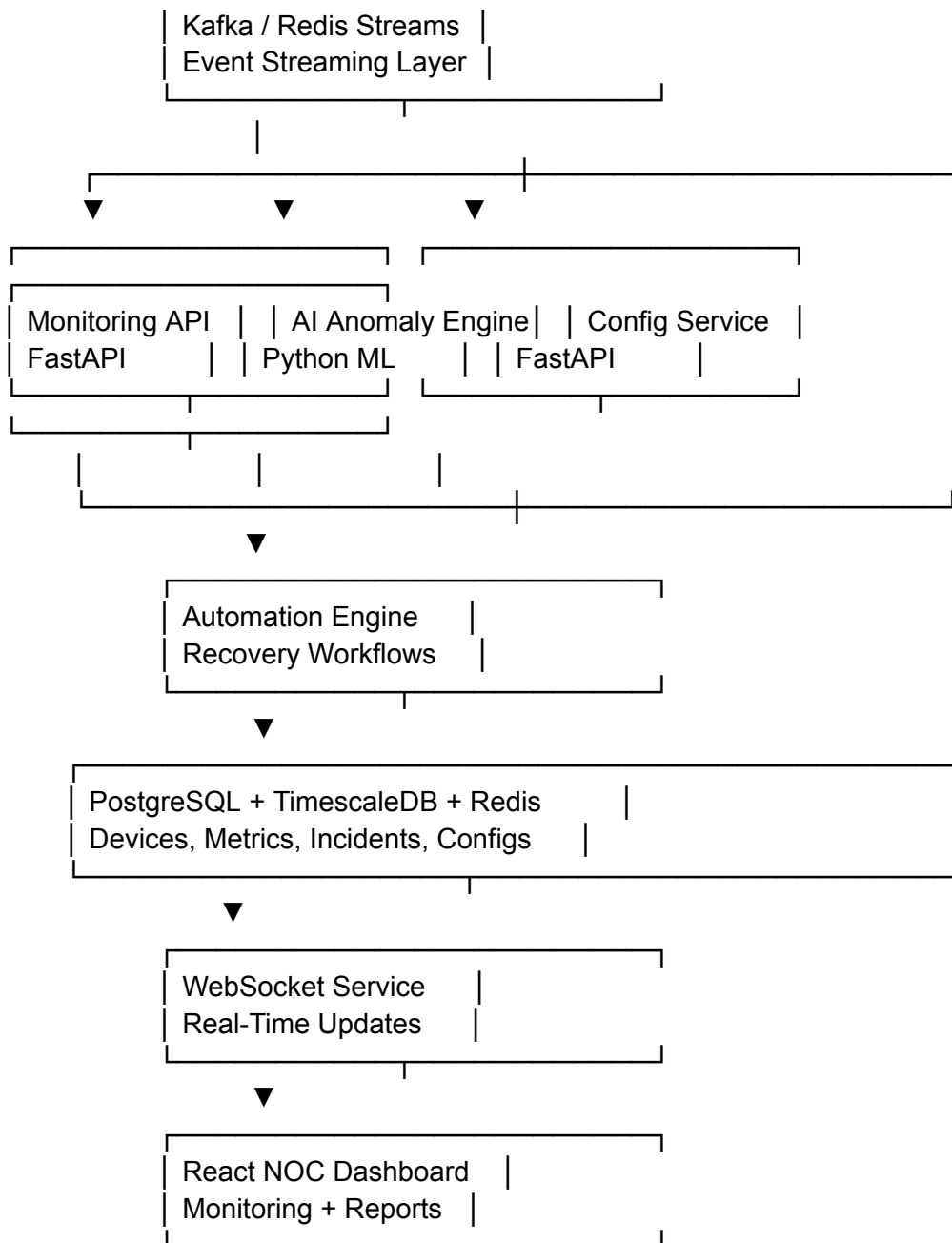
12. Observability

Add:

- Prometheus metrics
 - Grafana dashboards
 - structured logs
 - OpenTelemetry tracing
 - API latency monitoring
 - worker health monitoring
-

Complete Architecture





Recommended Tech Stack

Layer	Tech
Backend	Python, FastAPI

Streaming	Kafka or Redis Streams
Async Jobs	Celery
Database	PostgreSQL, TimescaleDB
Cache	Redis
AI/ML	Scikit-learn
Dashboard	React, Tailwind
Real-time	WebSockets
Reports	ReportLab / WeasyPrint
Monitoring	Prometheus, Grafana
Deployment	Docker Compose
Testing	Pytest
